WELT 2 · DOPPELSTUNDE 14 · LEVEL 2.2

W2·2 – Die Rangliste: Sortieren

Datei: `AQ_W2_L2_Rangliste_Geruest.txt` (Level 2.2) · IDE-Level 2.2: Basis · Erweiterung · Experte · Im Lab: **Level 2.2** · Material: je Team 8 Spielkarten

Eine Highscore-Liste, in der Platz 1 irgendwo in der Mitte steht, will niemand sehen. Die Rangliste muss **sortiert** werden: absteigend nach Punkten, der Beste ganz oben an Index 0. Sortieren ist eine der am besten untersuchten Aufgaben der Informatik – denn in sortierten Daten findet man alles dramatisch schneller (mehr dazu in W2·3).

Heute lernst du drei klassische Verfahren kennen: **Selectionsort**, **Insertionsort** und **Bubblesort**. Im Abitur musst du sie am Ablauf erkennen, von Hand ausführen, ihre Arbeit zählen und von Hand implementieren.

Aufgabe 0: Sortieren ohne Computer

PAPIER ★

Jedes Team bekommt 8 Spielkarten mit verschiedenen Zahlenwerten. Mischt sie und legt sie **verdeckt** in einer Reihe aus. Verteilt die Rollen: **Vergleicher**, **Tauscher**, **Zähler** (Strichliste) und **Protokollant**.

Die strengen Regeln

- Es dürfen immer nur **zwei Karten gleichzeitig** aufgedeckt und verglichen werden. Danach werden sie wieder umgedreht. Jeder Vergleich ist ein Strich.
- Karten dürfen nur **paarweise vertauscht** werden – nicht herausgenommen und woanders eingeschoben. Jede Vertauschung ist ein Strich.
- Ziel: **absteigend** sortiert – die höchste Karte ganz links (Platz 1), wie in der Rangliste.

a) Sortiert die Karten. Notiert die Zahl der Vergleiche und der Vertauschungen.

b) Beschreibt euer Vorgehen so präzise, dass ein anderes Team es **ohne Nachfragen** ausführen könnte. Genau das ist ein Algorithmus.

c) Sammelt die Ergebnisse der Teams an der Tafel.

Team	Vergleiche	Vertauschungen	Idee des Verfahrens in Stichworten

d) Welches Team brauchte die wenigsten Vergleiche – Zufall oder Verfahren? Formuliert eine Vermutung.

Hinweis: Fast alle Verfahren, die Menschen spontan erfinden, sind Varianten der drei Verfahren auf diesem Blatt. Vergleicht am Ende: Welches war eures?

Infoteil 1: Die zwei Werkzeuge – besser und vertausche

Jedes Sortierverfahren besteht aus nur zwei Grundoperationen: **vergleichen** und **vertauschen**. Im Gerüst sind beide als Methoden der Rangliste vorgegeben – und beide **zählen mit**:

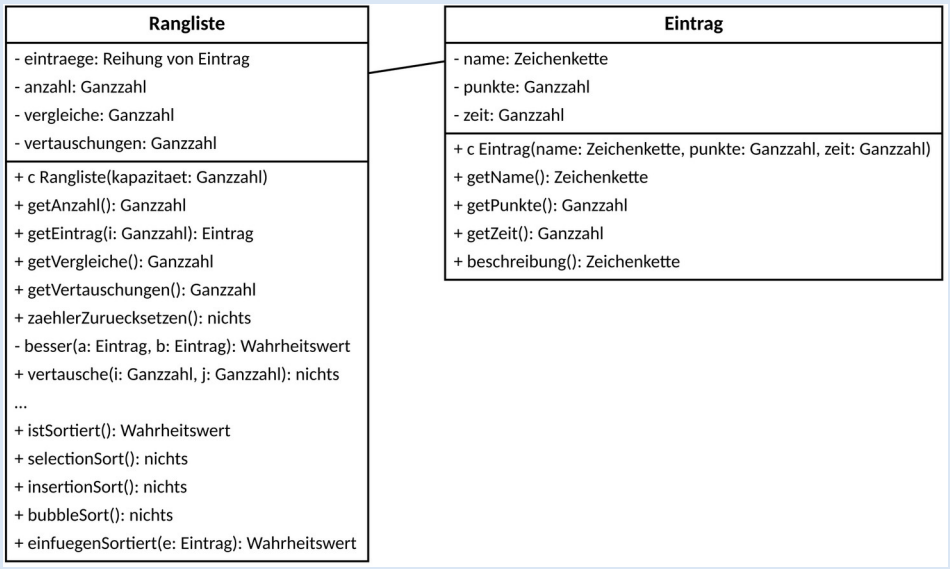


Abbildung 1: Klassenkarten (Ausschnitt bis Level 2.2)

```
private boolean besser(Eintrag a, Eintrag b) {
    vergleiche++;                          // jeder Aufruf = 1 Vergleich
    return a.getPunkte() > b.getPunkte();  // a gehoert weiter nach vorne
}

public void vertausche(int i, int j) {
    Eintrag hilf = eintraege[i];           // Dreieckstausch:
    eintraege[i] = eintraege[j];           // erst retten,
    eintraege[j] = hilf;                   // dann ueberschreiben
    vertauschungen++;                      // jeder Aufruf = 1 Vertauschung
}
```

Zähl-Regeln für Level 2.2: Beim Sortieren und sortierten Einfügen vergleichst du Einträge **nur** mit `besser(…)`, die drei Sortierverfahren tauschen **nur** mit `vertausche(…)`. Nur dann stimmen deine Zählerstände mit den Tests überein. `besser` ist **strikt** (>): Gleich gute Einträge sind nicht „besser“.

Aufgabe 1: Papier-Warm-up: Der Dreieckstausch

PAPIER ★

Die Rangliste enthält Aria, Brann, Cyra, Dorn (Index 0 bis 3). Ausgeführt wird `vertausche(1, 3)`.

a) Gib nach jeder der drei Tausch-Zeilen an, auf welchen Eintrag `hilf`, `eintraege[1]` und `eintraege[3]` verweisen.

nach Zeile	hilf	eintraege[1]	eintraege[3]
Eintrag hilf = eintraege[i];			
eintraege[i] = eintraege[j];			
eintraege[j] = hilf;			

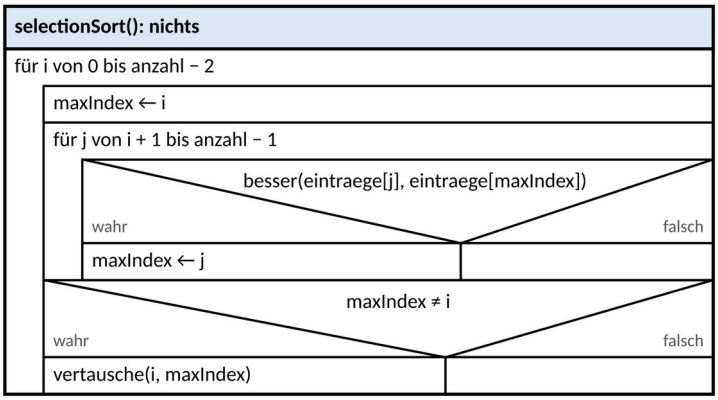
b) Eine Mitschülerin spart sich die Hilfsvariable: `eintraege[i] = eintraege[j]; eintraege[j] = eintraege[i];` Gib an, was danach in der Rangliste steht, und erläutere, was mit Brann passiert ist.

Infoteil 2: Selectionsort – Sortieren durch Auswählen

- **Idee:** Suche im unsortierten Rest den **besten** Eintrag (Baustein Maximum mit Index!) und tausche ihn an den Anfang des Restes. Danach ist der Rest um eins kürzer.
- Nach dem Durchlauf mit i steht an Index i der endgültig richtige Eintrag. Für die letzte Position bleibt nur einer übrig – darum läuft i nur bis **anzahl − 2**.
- Getauscht wird höchstens einmal pro Durchlauf – und nur, wenn der Beste nicht schon an Position i steht.

Beispiel (Punkte; grün = endgültig sortiert):

	Index 0	Index 1	Index 2	Index 3	Index 4	Bemerkung
Start	50	20	90	10	60	
nach i = 0	90	20	50	10	60	90 ist der Beste, tauscht mit 50
nach i = 1	90	60	50	10	20	60 tauscht mit 20
nach i = 2	90	60	50	10	20	50 steht schon richtig: kein Tausch
nach i = 3	90	60	50	20	10	20 tauscht mit 10 – fertig




```
public void selectionSort() {
    for (int i = 0; i < anzahl - 1; i++) {
        int maxIndex = i;
        for (int j = i + 1; j < anzahl; j++) {
            if (besser(eintraege[j], eintraege[maxIndex])) {
                maxIndex = j;
            }
        }
        if (maxIndex != i) {
            vertausche(i, maxIndex);
        }   // → Tabellenzeile
    }
}
```

Tipp: Profis zerlegen: Eine Hilfsmethode `indexDesBesten(ab: Ganzzahl): Ganzzahl` sucht den besten Eintrag im Bereich ab … anzahl − 1. Dann besteht selectionSort nur noch aus einer Schleife mit

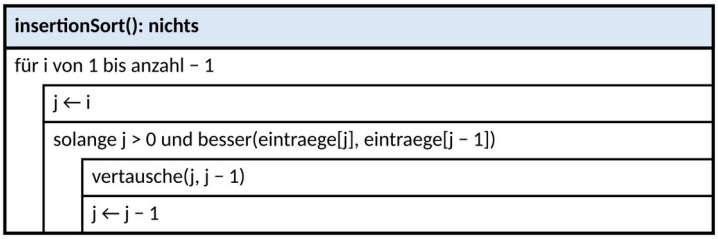
einem Aufruf und einem Tausch. Erst kleine Bausteine, dann zusammensetzen – diese Technik hilft dir auch im Abitur.

Infoteil 3: Insertionsort – Sortieren durch Einfügen

- **Idee:** Wie beim Aufnehmen von Spielkarten. Der linke Teil (anfangs nur Index 0) ist sortiert. Der nächste Eintrag wandert durch Tauschen mit dem linken Nachbarn **so lange nach links, wie er besser ist** als dieser.
- Die innere Schleife hat **zwei** Abbruchgründe: Der Eintrag ist ganz vorne angekommen (j = 0) oder der linke Nachbar ist nicht schlechter.
- Gleich gute Einträge werden nicht getauscht (besser ist strikt).

Beispiel (grün = der bereits sortierte linke Teil):

	Index 0	Index 1	Index 2	Index 3	Index 4	Bemerkung
Start	50	20	90	10	60	
nach i = 1	50	20	90	10	60	20 bleibt (50 ist besser)
nach i = 2	90	50	20	10	60	90 wandert ganz nach vorne
nach i = 3	90	50	20	10	60	10 bleibt
nach i = 4	90	60	50	20	10	60 wandert zwei Plätze nach links



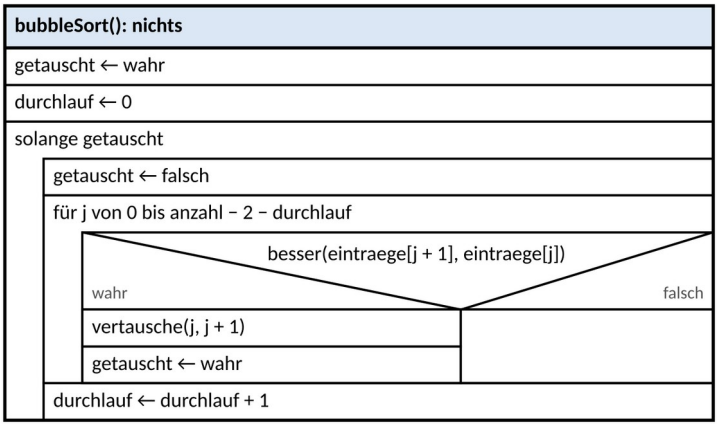

```
public void insertionSort() {
    for (int i = 1; i < anzahl; i++) {
        int j = i;
        while (j > 0 && besser(eintraege[j], eintraege[j - 1])) {
            vertausche(j, j - 1);
            j--;
        }   // → Tabellenzeile
    }
}
```

Infoteil 4: Bubblesort – Sortieren durch Aufsteigen

- **Idee:** Vergleiche immer zwei **Nachbarn** und tausche sie, wenn der rechte besser ist. Nach einem kompletten Durchlauf ist der schlechteste Eintrag ganz hinten angekommen – er ist wie eine Blase nach unten „gesunken", die guten steigen nach oben.
- Nach jedem Durchlauf ist hinten ein Platz mehr fertig: Die innere Schleife wird **um 1 kürzer**.
- **Abbruch:** Gab es in einem Durchlauf keine einzige Vertauschung, ist die Liste sortiert – das Verfahren endet.

Beispiel (grün = hinten bereits endgültig):

	Index 0	Index 1	Index 2	Index 3	Index 4	Bemerkung
Start	50	20	90	10	60	
Durchlauf 1	50	90	20	60	10	4 Vergleiche, 2 Tausche
Durchlauf 2	90	50	60	20	10	3 Vergleiche, 2 Tausche
Durchlauf 3	90	60	50	20	10	2 Vergleiche, 1 Tausch
Durchlauf 4	90	60	50	20	10	1 Vergleich, kein Tausch → Ende



Java – zunächst **ohne** Abbruch (die innere Schleife wird schon kürzer). Den Abbruch mit `getauscht` baust du in Level 2.2 selbst ein – das Struktogramm zeigt, wie.

```
public void bubbleSort() {
    for (int durchlauf = 0; durchlauf < anzahl - 1; durchlauf++) {
        for (int j = 0; j < anzahl - 1 - durchlauf; j++) {
            if (besser(eintraege[j + 1], eintraege[j])) {
                vertausche(j, j + 1);
            }
        }   // → Tabellenzeile
    }
}
```

Achtung Abi: Sortierverfahren von Hand

- „Zustand nach jedem Durchlauf der **äußeren** Schleife" heißt: eine Zeile pro Durchlauf – nicht pro Vertauschung.
- Insertionsort in Java: `j > 0` muss **vor** dem Vergleich stehen. `&&` wertet von links aus – sonst wird `eintraege[-1]` angefasst.
- Erkennungsmerkmale: Selectionsort sucht ein Extremum und tauscht **weit**; Insertionsort lässt einen Eintrag **nach links wandern**; Bubblesort vergleicht **nur Nachbarn** über die ganze (restliche) Liste.
- Absteigend oder aufsteigend? Lies die Aufgabe genau – oft unterscheidet sich nur ein Vergleichszeichen.

Aufgabe 2: Drei Verfahren, eine Rangliste – Trace

PAPIER ★★

Fünf Einträge A bis E stehen mit diesen Punkten in der Rangliste: **50, 80, 30, 90, 70**. Sortiert wird absteigend. Gib jeweils den Zustand nach jedem Durchlauf der äußeren Schleife an und zähle **Aufrufe von besser** und **von vertausche** mit (fortlaufende Summe).

a) Selectionsort (Infoteil 2) – eine Zeile an der mit „// → Tabellenzeile“ markierten Stelle: **[3 BE]**

nach …	Index 0	Index 1	Index 2	Index 3	Index 4	Vergleiche (Summe)	Vertauschungen (Summe)
Start	50	80	30	90	70	0	0
i = 0							
i = 1							
i = 2							
i = 3							

b) Insertionsort (Infoteil 3) – eine Zeile an der mit „// → Tabellenzeile“ markierten Stelle: **[3 BE]**

nach …	Index 0	Index 1	Index 2	Index 3	Index 4	Vergleiche (Summe)	Vertauschungen (Summe)
Start	50	80	30	90	70	0	0
i = 1							
i = 2							
i = 3							
i = 4							

c) Bubblesort **mit Abbruch** (Struktogramm Infoteil 4) – Zeile je Durchlauf: **[3 BE]**

nach …	Index 0	Index 1	Index 2	Index 3	Index 4	Vergleiche (Summe)	Vertauschungen (Summe)
Start	50	80	30	90	70	0	0
Durchlauf 1							
Durchlauf 2							
Durchlauf 3							
Durchlauf 4							

d) Vergleiche die drei Verfahren: Welches vertauscht am wenigsten, welches vergleicht am wenigsten? Begründe mit dem Aufbau der Verfahren. **[3 BE]**

Kontrolle: Der Test `experte_insertionSort_traceZaehlen` in Level 2.2 benutzt genau diese Liste.

Aufgabe 3: Die Rangliste ordnet sich – Level 2.2, Teil 1

IDE ★ – ★★★

a) Implementiere `istSortiert()`: wahr, wenn die Liste absteigend sortiert ist (gleiche Nachbarn erlaubt). Vergleiche hier **direkt** mit `getPunkte()` – istSortiert soll nicht mitzählen. Achte auf das letzte Nachbarpaar.

b) Implementiere `selectionSort()` und `insertionSort()` nach den Infoteilen und `bubbleSort()` **mit Abbruch** nach dem Struktogramm.

c) **Vorhersage ohne Gerät:** Das Hauptprogramm sortiert die Demo-Liste (1200, 800, 1500, 950, 600, 1100) mit insertionSort und gibt die Zählerstände aus. Wie viele Vergleiche und Vertauschungen erwartest du? Erst notieren, dann starten.

d) Starte `L22SortierTest`. Die Tests prüfen das Ergebnis **und** die genauen Zählerstände – z. B. im besten Fall (schon sortiert) genau n − 1 Vergleiche bei Insertions- und Bubblesort.

Stufe	Tests in L22SortierTest beginnen mit …
★ Basis	`basis_`
★★ Erweiterung	`erweiterung_`
★★★ Experte	`experte_`

e) **Stopp:** `einfuegenSortiert` folgt in Aufgabe 6. Bis dahin bleiben dessen Tests rot.

Tipp: Stimmt die Zahl der Vergleiche nicht? Dann vergleichst du irgendwo ohne `besser` – oder deine innere Schleife bricht nicht ab bzw. wird nicht kürzer. Die Meldung des Tests nennt die erwartete Zahl.

Aufgabe 4: Gleichstand! – Stabilität

PAPIER ★★

Die Rangliste speichert die Einträge in der Reihenfolge, in der die Läufe beendet wurden. Bei Punktgleichstand soll der Eintrag vorne stehen, der die Punktzahl **zuerst** erreicht hat.

Index	0	1	2	3
Eintrag	Aria 500	Brann 800	Cyra 500	Dorn 800

a) Gib das Ergebnis von **Selectionsort** und von **Insertionsort** an (nur die Namen). **[2 BE]**

b) Ein Sortierverfahren heißt **stabil**, wenn gleich gute Einträge ihre ursprüngliche Reihenfolge behalten. Entscheide für beide Verfahren, ob sie hier stabil arbeiten, und erläutere, welcher Schritt von Selectionsort die Reihenfolge von Aria und Cyra verändert. **[3 BE]**

Denkanstoß: Ist Bubblesort stabil? Und was müsste man an Insertionsort ändern, damit es nicht mehr stabil ist? (Ein einziges Zeichen genügt.)

Aufgabe 5: Abi-Training: Verfahren erkennen

PAPIER ★★★

Gegeben ist die Methode `sortiere` einer Hilfsklasse. Sie arbeitet auf einer Reihung von Ganzzahlen. Bearbeitungszeit wie im Abitur: ca. 2,5 Minuten pro BE.

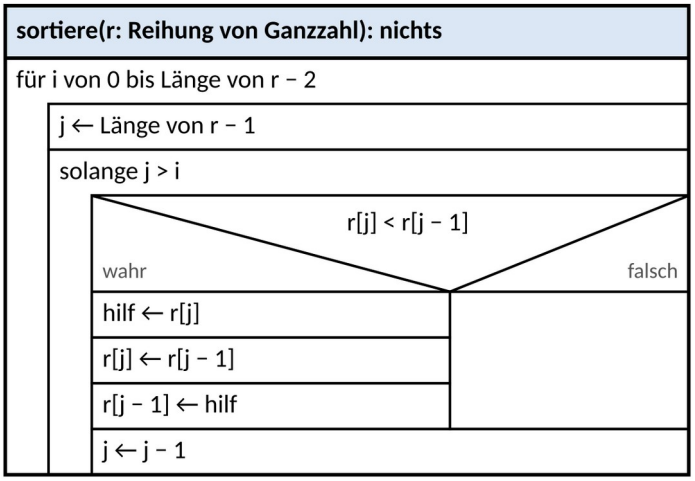


Abbildung 2: Die Methode sortiere

a) Gib den Zustand der Reihung r = {7, 3, 9, 1, 5} nach jedem Durchlauf der äußeren Schleife an. **[3 BE]**

nach ...	r[0]	r[1]	r[2]	r[3]	r[4]
i = 0					
i = 1					
i = 2					
i = 3					

b) Gib an, um welches der drei Sortierverfahren es sich handelt, und erläutere die Funktionsweise der Methode. Gehe dabei auf die Rolle der beiden Schleifen ein. **[5 BE]**

c) Gib an, wie oft die Bedingung r[j] < r[j − 1] für die Reihung aus a) geprüft wird und wie viele Vertauschungen stattfinden. **[2 BE]**

d) Gib an, wie die Methode geändert werden muss, damit sie absteigend sortiert. **[1 BE]**

Infoteil 5: Sortiert einfügen

Ist die Rangliste schon sortiert und kommt **ein** neuer Eintrag hinzu, muss nicht neu sortiert werden. Es genügt ein einziger Einfüge-Schritt – wie bei Insertionsort, aber mit **Verschieben** statt Tauschen:

- Prüfe zuerst: Ist e null oder die Liste voll? Dann false.
- Beginne hinter dem letzten Eintrag (i = anzahl). Solange der linke Nachbar **schlechter** ist als e, schiebe ihn eine Stelle nach rechts: `eintraege[i] = eintraege[i - 1]`.
- In die entstandene Lücke kommt e; anzahl wird erhöht. Bei Gleichstand stoppt das Schieben – e landet **hinter** gleich guten Einträgen (stabil).

Beispiel: Gwen (1000 Punkte) kommt in die Rangliste mit Kapazität 6:

	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Bemerkung
vorher	Cyra 1500	Aria 1200	Dorn 950	Brann 800	null	null	
1. Vergleich	Cyra 1500	Aria 1200	Dorn 950	Brann 800	Brann 800	null	Gwen besser als Brann: schieben
2. Vergleich	Cyra 1500	Aria 1200	Dorn 950	Dorn 950	Brann 800	null	Gwen besser als Dorn: schieben
3. Vergleich	Cyra 1500	Aria 1200	Gwen 1000	Dorn 950	Brann 800	null	nicht besser als Aria: Gwen in die Lücke

3 Vergleiche, 0 Vertauschungen. Zwischendurch steht ein Eintrag **doppelt** in der Reihung – das ist gewollt, er wird gleich überschrieben.

Aufgabe 6: Neu in der Rangliste – sortiert einfügen

PAPIER → IDE ★★

a) Hugo (950 Punkte) wird in die Rangliste **vor** Gwens Ankunft eingefügt (Cyra 1500, Aria 1200, Dorn 950, Brann 800). Gib die Reihenfolge danach und die Zahl der Vergleiche an. Begründe, warum Hugo hinter Dorn landet. **[2 BE]**

b) Ergänze die Lücken – ohne Gerät. **[4 BE]**

```
1  public boolean einfuegenSortiert(Eintrag e) {
2      if (_____________________________________) {
3          return false;
4      }
5      int i = __________;
6      while (i > 0 && _____________________________________) {
7          eintraege[i] = ____________________;
8          i--;
9      }
10     ____________________ = e;
11     ____________________;
12     return true;
13 }
```

c) Tippe deine Lösung in die IDE ab und starte `L22SortierTest`. Jetzt sollten auch alle Tests zu einfuegenSortiert grün werden – auch `experte_einfuegenSortiert_ohneVertausche`.

d) Erläutere, warum man fünf Einträge auch sortieren kann, indem man sie nacheinander mit `einfuegenSortiert` in eine **leere** Liste einfügt. Welches Sortierverfahren steckt dahinter? **[2 BE]**

🎯 Im Lab üben: Level 2.2 „Ordnung im Ranking – Selection-, Insertion-, Bubblesort“ – Punkte gibt es dort automatisch

Das Lab-Level 2.2 im **Algo Quest Lab** (algoquest.frankjuchim.de) passt zu diesem Blatt. Die Aufgaben werden automatisch geprüft, Punkte gibt es beim ersten Lösen. Das IDE-Level 2.2 programmierst du weiterhin in der Online-IDE.

Zum Festigen: Der **Trainingsplatz** von Welt 2 stellt Zufallsaufgaben – je ein Durchlauf Bubble-, Selection- oder Insertionsort.

Für Schnelle: Der dümmste Sortieralgorithmus der Welt

BogoSort mischt die Liste zufällig und prüft mit `istSortiert()`, ob sie zufällig sortiert ist. Falls nicht: wieder mischen. Und wieder …

- **Denkanstoß:** Wie viele verschiedene Reihenfolgen gibt es für 5 verschiedene Einträge? Wie viele Mischversuche braucht BogoSort dann ungefähr im Mittel? Und bei 10 und bei 20 Einträgen? Schätze ab, wie lange 20 Einträge bei 100 Millionen Versuchen pro Sekunde dauern.
- Implementiere `bogoSort()` in der Rangliste: Mischen mit `vertausche(i, Zufallsindex)` für jede Position i, Zufallsindex mit `(int) (Math.random() * anzahl)`. Zähle die Versuche. Probiere 5, dann 8 Einträge (nicht mehr!).
- Sortiere die Rangliste **alphabetisch nach Namen** mit Insertionsort: Vergleich mit `compareTo`. Warum geht `<` bei Zeichenketten nicht?

Side-Quest: freiwillig und ohne Punkte.